
CIENCIA DE DATOS / Unidad 2 / 2.3

Preprocesamiento de datos

El puente entre los datos crudos y el modelo: **rellenar lo que falta, convertir categorías en números, poner todo en la misma escala** y **encadenar esos pasos** de forma reproducible y sin fugas de información.

- 2.3.1 Tratamiento de valores faltantes
- 2.3.2 Codificación de variables categóricas
- 2.3.3 Escalado y normalización
- 2.3.4 Pipelines de transformación

2.3 Preprocesamiento de datos

El análisis exploratorio nos dice cómo son los datos; el **preprocesamiento** los deja listos para un modelo. La mayoría de los algoritmos de machine learning imponen condiciones estrictas sobre su entrada: no admiten valores faltantes, solo entienden números (no texto), y muchos son sensibles a que las variables estén en escalas muy distintas. Preprocesar es, precisamente, el trabajo de satisfacer esas condiciones sin distorsionar la información.

CLAVE

Garbage in, garbage out. Ningún algoritmo, por sofisticado que sea, compensa un preprocesamiento descuidado. La calidad del resultado está limitada por la calidad de los datos que entran.

En Python, la biblioteca de referencia para estas tareas es **scikit-learn** (que se importa como `sklearn`). Ofrece objetos *transformadores* con una interfaz uniforme de dos métodos que usaremos todo el tiempo:

- **.fit(X)** — *aprende* los parámetros necesarios a partir de los datos (por ejemplo, la media de una columna, o las categorías presentes).
- **.transform(X)** — *aplica* esa transformación y devuelve los datos convertidos. `.fit_transform(X)` hace ambos pasos de una vez.

Usaremos un mismo conjunto de ejemplo —datos de clientes— a lo largo de las cuatro subsecciones, para ver cómo se encadenan los pasos. Fíjate en los valores faltantes marcados en rojo:

edad	ciudad	educacion	salario	compro
25	La Paz	Secundaria	4200	no
NaN	Potosí	Universidad	7800	si
41	La Paz	Primaria	NaN	no
33	Sucre	Universidad	9100	si
52	Potosí	Secundaria	5400	no

NOTA

Este conjunto mezcla los tres retos típicos: valores faltantes (edad, salario), variables categóricas (ciudad, educacion) y variables numéricas en escalas distintas (edad vs. salario). Los abordaremos en ese orden.

2.3.1 Tratamiento de valores faltantes

Un valor faltante (*missing value*, representado como NaN) es un dato que no fue registrado: un formulario incompleto, un sensor que falló, una respuesta omitida. La mayoría de los algoritmos no toleran NaN y lanzan un error, así que hay que resolverlos antes de entrenar.

Primero: detectar y medir

Antes de decidir qué hacer, conviene saber cuántos faltantes hay y dónde:

```
In [1]:
df.isna().sum()          # faltantes por columna
df.isna().mean() * 100   # porcentaje de faltantes por columna

Out[1]:
edad      1
salario    1
dtype: int64
```

Estrategia 1 — Eliminación

Descartar los datos incompletos. Es simple, pero se pierde información; solo es razonable cuando los faltantes son **pocos** o cuando una columna tiene tantos huecos que resulta inservible.

```
In [2]:
df.dropna()              # elimina filas con algún NaN
df.dropna(axis=1)         # elimina columnas con algún NaN
df.dropna(subset=['edad']) # solo si falta 'edad'
df.dropna(thresh=4)       # conserva filas con >= 4 valores no nulos
```

ATENCIÓN

El riesgo de dropna(). Eliminar filas puede sesgar los datos si los faltantes no son aleatorios (por ejemplo, si las personas de mayores ingresos tienden a no declarar su salario). Descartar reduce la muestra y puede eliminar justo el patrón que buscabas.

Estrategia 2 — Imputación

DEFINICIÓN

Imputación. Reemplazar cada valor faltante por una estimación razonable, conservando la fila. Para variables numéricas se usa típicamente la *media* o la *mediana*; para categóricas, la *moda* (el valor más frecuente).

¿Media o mediana? La **mediana** es más robusta cuando hay valores atípicos, porque no se deja arrastrar por ellos. Con Pandas, la imputación directa es:

```
In [3]:
df['edad'].fillna(df['edad'].median(), inplace=True)
df['salario'].fillna(df['salario'].median(), inplace=True)
df['ciudad'].fillna(df['ciudad'].mode()[0], inplace=True) # moda
```

scikit-learn ofrece [SimpleImputer](#), que hace lo mismo pero como transformador reutilizable —clave para los pipelines de la sección 2.3.4—:

```
In [4]:  
from sklearn.impute import SimpleImputer  
  
imp = SimpleImputer(strategy='median')  
df[['edad', 'salario']] = imp.fit_transform(df[['edad', 'salario']])
```

EJEMPLO

Qué acaba de pasar. En `.fit()` el imputador calculó y guardó la mediana de cada columna (33.0 y 5400.0). En `.transform()` rellenó cada NaN con esa mediana. El NaN de edad se vuelve 33.0; el de salario, 5400.0.

Existen métodos más sofisticados —`KNNImputer` (usa las filas más parecidas) o `IterativeImputer` (modela cada columna a partir de las demás)— útiles cuando la imputación simple distorsiona demasiado la distribución.

2.3.2 Codificación de variables categóricas

Los algoritmos operan con números, pero muchas columnas son **texto**: una ciudad, un nivel de estudios, un color. **Codificar** es convertir esas categorías en números —pero no de cualquier forma—. La elección correcta depende de un matiz importante: si la variable tiene un orden natural o no.

DEFINICIÓN

Variable nominal. Categorías sin orden: ciudad (La Paz, Potosí, Sucre). Ninguna es "mayor" que otra.

Variable ordinal. Categorías con orden intrínseco: educacion (Primaria < Secundaria < Universidad).

Codificación ordinal (Label / Ordinal Encoding)

Asigna un entero a cada categoría según su orden: Primaria→0, Secundaria→1, Universidad→2. Es correcta **solo** cuando ese orden tiene sentido, porque el número transmite magnitud (2 es "más" que 0).

In [5]:

```
from sklearn.preprocessing import OrdinalEncoder

orden = [['Primaria', 'Secundaria', 'Universidad']]
enc = OrdinalEncoder(categories=orden)
df['educacion'] = enc.fit_transform(df[['educacion']])
# Primaria->0.0, Secundaria->1.0, Universidad->2.0
```

ATENCIÓN

El error más común. Aplicar codificación ordinal a una variable *nominal*. Si codificas La Paz→0, Potosí→1, Sucre→2, el modelo asumirá que Sucre > Potosí > La Paz y que la "distancia" La Paz–Sucre es el doble que La Paz–Potosí. Ese orden es falso e introduce ruido. Para variables nominales, usa One-Hot.

Codificación One-Hot

DEFINICIÓN

One-Hot Encoding. Crea una columna binaria (0/1) por cada categoría. Cada fila tiene un 1 en la columna de su categoría y 0 en las demás. No impone ningún orden, por lo que es la opción correcta para variables nominales.

La forma más directa en Pandas es `pd.get_dummies`:

In [6]:

```
pd.get_dummies(df, columns=['ciudad'])
```

La columna ciudad se expande en tres columnas indicadoras:

edad	ciudad_La Paz	ciudad_Potosí	ciudad_Sucre
25	1	0	0

edad	ciudad_La Paz	ciudad_Potosí	ciudad_Sucre
33	0	1	0
41	1	0	0
33	0	0	1

NOTA

La trampa de las dummy. Las tres columnas son redundantes: si dos son 0, la tercera es forzosamente 1. En modelos lineales esto causa problemas de colinealidad. Se resuelve descartando una columna con `drop_first=True` (o `drop='first'` en `OneHotEncoder`).

El equivalente en scikit-learn es [OneHotEncoder](#), preferible dentro de un pipeline porque "recuerda" las categorías vistas en el entrenamiento y las aplica igual a datos nuevos:

In [7]:

```
from sklearn.preprocessing import OneHotEncoder

ohe = OneHotEncoder(drop='first', sparse_output=False)
codificado = ohe.fit_transform(df[['ciudad']])
```

CLAVE

Regla práctica: ¿la variable tiene orden natural? → **Ordinal**. ¿No lo tiene? → **One-Hot**. ¿Tiene cientos de categorías (p. ej. códigos postales)? → conviene explorar técnicas específicas para alta cardinalidad, no One-Hot directo.

2.3.3 Escalado y normalización

En nuestro dataset, edad va de 25 a 52, mientras que salario va de 4200 a 9100. Muchos algoritmos —los que miden distancias (KNN, K-Means, SVM) y los que usan descenso de gradiente (regresión, redes neuronales)— darían un peso desproporcionado al salario solo porque sus números son más grandes. **Escalar** pone todas las variables numéricas en un rango comparable para que ninguna domine por su magnitud.

Normalización (Min-Max)

Reescala cada variable al rango [0, 1]. El mínimo se vuelve 0, el máximo 1 y el resto se distribuye proporcionalmente.

$$x_{\text{norm}} = (x - \min) / (\max - \min)$$

```
In [8]:  
from sklearn.preprocessing import MinMaxScaler  
  
mm = MinMaxScaler()  
df[['edad', 'salario']] = mm.fit_transform(df[['edad', 'salario']])
```

Es intuitiva y acota los valores, pero es sensible a atípicos: un único valor extremo comprime a todos los demás cerca de 0.

Estandarización (Z-score)

DEFINICIÓN

Estandarización. Transforma cada variable para que tenga media 0 y desviación estándar 1. No acota a un rango fijo, pero es más robusta a atípicos que Min-Max y es la opción por defecto en la mayoría de casos.

$$z = (x - \mu) / \sigma \quad \mu = \text{media}, \quad \sigma = \text{desviación estándar}$$

```
In [9]:  
from sklearn.preprocessing import StandardScaler  
  
sc = StandardScaler()  
df[['edad', 'salario']] = sc.fit_transform(df[['edad', 'salario']])
```

Un valor $z = 1.5$ significa "1.5 desviaciones por encima de la media"; $z = -0.5$, "media desviación por debajo". Existe además `RobustScaler`, que usa la mediana y el rango intercuartílico, ideal cuando hay muchos atípicos.

La regla de oro: evitar la fuga de datos

ATENCIÓN

Data leakage (fuga de información). Es el error más grave del preprocesamiento y también el más silencioso, porque no produce ningún mensaje de error: el modelo simplemente parece mejor de lo que es.

El escalador debe **aprender** sus parámetros (mín, máx, media, σ) **únicamente del conjunto de entrenamiento**, y luego aplicarse tanto a entrenamiento como a prueba. Calcularlos con todos los datos deja que información del conjunto de prueba se filtre al modelo.

In [10]:

```
from sklearn.model_selection import train_test_split

X_train, X_test = train_test_split(X, test_size=0.2, random_state=42)

sc = StandardScaler()
X_train = sc.fit_transform(X_train) # fit + transform SOLO en train
X_test = sc.transform(X_test)      # transform en test (sin re-fit)
```

CLAVE

fit_transform en entrenamiento; **transform** —a secas— en prueba. Nunca llames a fit sobre los datos de prueba. Esta disciplina es la razón principal para usar pipelines.

2.3.4 Pipelines de transformación

Hasta aquí aplicamos los pasos uno por uno. En un proyecto real esto se vuelve frágil: hay que recordar el orden, repetir exactamente las mismas transformaciones sobre los datos de prueba y sobre los datos nuevos en producción, y no equivocarse con la fuga de datos. Un **pipeline** resuelve todo eso encadenando los pasos en un solo objeto.

DEFINICIÓN

Pipeline. Secuencia de transformadores (y, opcionalmente, un modelo final) tratada como una sola unidad. Al llamar `.fit()` sobre el pipeline, cada paso aprende en orden; al llamar `.transform()` o `.predict()`, se aplican en cadena. Un solo objeto, reproducible y a prueba de fugas.

Las ventajas son concretas:

- **Reproducibilidad:** la misma secuencia se aplica idéntica a train, test y datos futuros.
- **Sin fugas:** al hacer `fit` solo sobre el entrenamiento, todos los pasos aprenden únicamente de él, automáticamente.
- **Menos errores:** desaparece el riesgo de olvidar un paso o alterar el orden.

Un pipeline simple

Encadenar imputación y escalado para variables numéricas se ve así:

```
In [11]:
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler

pipe_num = Pipeline([
    ('imputar', SimpleImputer(strategy='median')),
    ('escalar', StandardScaler()),
])

X_train = pipe_num.fit_transform(X_train) # imputa y escala en cadena
X_test = pipe_num.transform(X_test)      # misma cadena, sin re-fit
```

ColumnTransformer: distinto trato por tipo de columna

Nuestro dataset necesita tratamientos **diferentes** según la columna: las numéricas (edad, salario) se imputan y escalan; las categóricas (ciudad, educacion) se imputan y codifican. `ColumnTransformer` aplica cada sub-pipeline a las columnas que le indiques y une los resultados:

```
In [12]:
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder

num = ['edad', 'salario']
cat = ['ciudad', 'educacion']
```

```
pipe_num = Pipeline([
    ('imp', SimpleImputer(strategy='median')),
    ('esc', StandardScaler()),
])
pipe_cat = Pipeline([
    ('imp', SimpleImputer(strategy='most_frequent')),
    ('ohe', OneHotEncoder(handle_unknown='ignore')),
])

preprocesador = ColumnTransformer([
    ('numericas', pipe_num, num),
    ('categoricas', pipe_cat, cat),
])
```

EJEMPLO

Lectura del bloque. Se definen dos mini-pipelines (uno numérico, uno categórico) y el ColumnTransformer enruta cada grupo de columnas al suyo. `handle_unknown='ignore'` evita que el modelo falle si en producción aparece una ciudad no vista en el entrenamiento.

El preprocesador completo, en dos líneas

Una vez definido, todo el preprocesamiento se aplica —correctamente y sin fugas— con la interfaz de siempre:

```
In [13]:
X_train = preprocesador.fit_transform(X_train) # aprende y aplica
X_test  = preprocesador.transform(X_test)      # solo aplica
```

CLAVE

El paso siguiente. El preprocesador puede convertirse en el primer paso de un pipeline mayor que termine en un modelo: `Pipeline(['prep', preprocesador], ('modelo', ...))`. Así, un único `.fit()` limpia, codifica, escala y entrena; y un único `.predict()` hace lo mismo con datos nuevos. Ese es el objetivo final del preprocesamiento ordenado.

Síntesis

El preprocesamiento sigue un orden lógico que ahora reconoces: **resolver faltantes** (imputar o eliminar) → **codificar categorías** (ordinal si hay orden, one-hot si no) → **escalar las numéricas** (estandarización por defecto) → **encadenar todo en un pipeline** que aprende solo del entrenamiento y se aplica idéntico a cualquier dato futuro. Un preprocesamiento así de disciplinado no es un trámite: es lo que hace que un modelo sea honesto, reproducible y confiable.